

CampusGuard

Emergency-response coordination platform for a university campus

COS 214 - Practical 5, 2026

Design, Implementation and Testing Documentation

Team member	Student number	Primary responsibility
Musa	U25098022	Incident core, State pattern, Mediator pattern, Docker/build
Ancheen	U25073703	Command pattern, response units, Observer wiring
Katlego	U24876489	Adapter pattern, Facade pattern, subsystems

Course / module	COS 214 - Software Modelling
Submission date	29/09/2026
Repository link	https://github.com/anchenk/COS_Prac5
Language / toolchain	C++11, g++, Make, Docker (Ubuntu 22.04), Valgrind, GDB

Patterns implemented: **Command, Mediator, Adapter, Facade, State, Observer**

Contents

1. System overview	3
1.1 Pattern map	3
1.2 How to run	3
2. Participants and contributions	4
2.1 Git contribution evidence	4
3. Full class diagram	5
4. Pattern design and justification	6
4.1 Command pattern	6
4.2 Mediator pattern (with the ResponseUnit hierarchy)	7
4.3 State pattern	8
4.4 Observer pattern	10
4.5 Adapter pattern	11
4.6 Facade pattern	12
5. How the patterns work together	14
6. Ownership and memory management	14
7. Invalid operations and error handling	15
8. Testing and verification	15
8.1 Automated tests	15
8.2 Demonstration scenarios	15
8.3 Valgrind (memory checking)	15
8.4 GDB (debugging)	16
8.5 Docker	16
9. Design decisions, assumptions and limitations	17
9.1 Decisions	17
9.2 Assumptions	17
9.3 Known limitations and possible extensions	17
10. Repository structure and build system	17
11. Requirements checklist	17
Appendix A - Captured demo output	19
Appendix B - Public interfaces	21

1. System overview

CampusGuard coordinates the response to emergencies on a university campus. An **Incident** (for example a fire in the Engineering Building) moves through a lifecycle. Operators issue commands (dispatch a unit, lock a zone, broadcast an alert, cancel the last action). **Response units** (security, medical, facilities) must react to each other without being tightly coupled. Alerts must reach an old **legacy siren gateway** as well as modern channels, and a single call must be able to start a full **evacuation**.

Each of these needs maps to one design pattern. The six patterns cooperate rather than sit side by side: a single operator command triggers the Command, Mediator, State and Observer patterns in one chain (section 5), and one facade call triggers the Facade, Adapter and Mediator patterns (section 5).

1.1 Pattern map

Pattern	Problem it solves in CampusGuard	Key classes	Owner
Command	Operator actions must be objects so they can be executed, recorded and undone.	Command, OperatorConsole, DispatchUnitCommand, LockdownZoneCommand, IssueAlertCommand, CancelActionCommand	Ancheen
Mediator	Response units must coordinate without knowing about each other.	IncidentMediator, IncidentCoordinator, ResponseUnit + 3 subclasses	Musa (mediator), Ancheen (units)
State	An incident behaves differently at each lifecycle stage; illegal transitions must be rejected.	Incident, IncidentState, Reported/Dispatched/Contained/ResolvedState	Musa
Observer	Several independent listeners must be told when an incident changes.	Incident (subject), IncidentObserver, IncidentLogger, IncidentDashboard	Ancheen
Adapter	The legacy gateway has an incompatible interface and must be usable as a modern channel.	INotificationChannel, LegacyGatewayAdapter, LegacySecurityGateway	Katlego
Facade	A multi-step evacuation should be one simple call over several subsystems.	EmergencyOperationsFacade, AccessControlSystem, AlertService	Katlego

1.2 How to run

- **Docker (required for the demonstration):** `docker compose up --build` - builds the image and runs the demo.
- **Local development:** `make, make run, make test, make valgrind, make clean`.
- The demo is non-interactive: it runs seven labelled scenarios (Appendix A) that exercise every pattern, including failure cases.

2. Participants and contributions

The work was split by pattern so each member owned a coherent vertical slice. Testing was a shared responsibility.

Member	Student no.	Responsibilities	Main files
Musa	U25098022	Incident core; State pattern (four concrete states, illegal-transition handling); Mediator (IncidentCoordinator, evacuation coordination); Dockerfile, docker-compose, Makefile and build system.	include/incident/*, src/incident/*, Dockerfile, docker-compose.yml, Makefile
Ancheen	U25073703	Command pattern (invoker with history and undo, four concrete commands); ResponseUnit hierarchy (security, medical, facilities); Observer wiring (IncidentObserver, logger, dashboard).	include/commands/*, include/units/*, include/observer/*, and matching src/
Katlego	U24876489	Adapter pattern (legacy gateway with awkward interface, translating adapter); Facade (EmergencyOperationsFacade); subsystems AccessControlSystem and AlertService.	include/adapter/*, include/facade/*, and matching src/
All		Integration in main.cpp, automated tests, Valgrind and GDB sessions, this document.	src/main.cpp, tests/*, docs/*

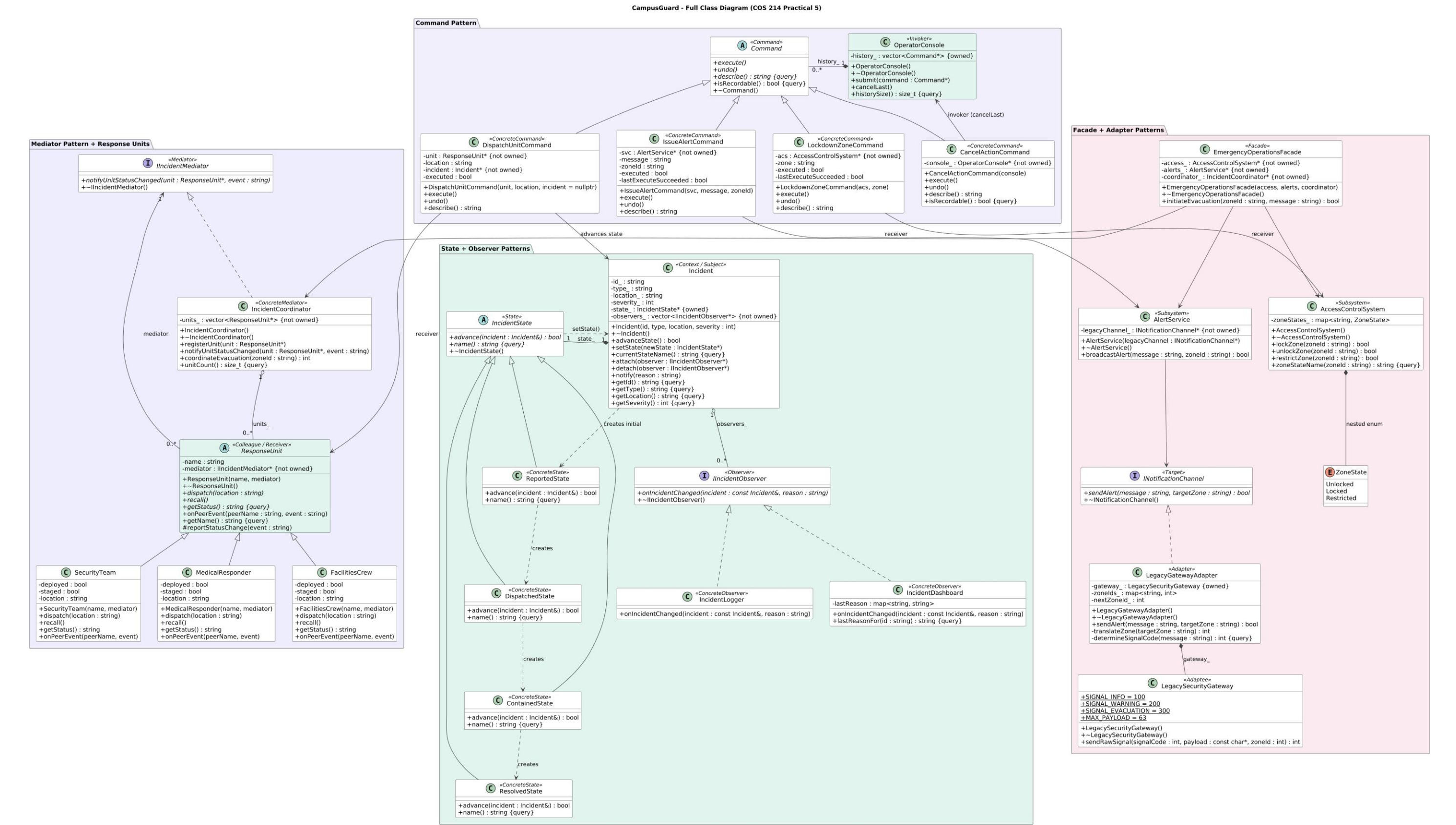
[PLACEHOLDER: confirm the split above matches what each member actually did, and add anything that changed (e.g. who wrote the tests, who ran GDB)]

2.1 Git contribution evidence



3. Full class diagram

All classes and relationships in one view. Source: docs/uml/campusguard_class_diagram.puml (paste into PlantUML to edit). Stereotypes show each class's role in its pattern. Composition (filled diamond) marks ownership; aggregation (hollow diamond) and plain arrows are non-owning references.



4. Pattern design and justification

Each subsection gives the intent, why the pattern was chosen over a simpler alternative, the mapping of pattern roles to classes, the class diagram, the behaviour visible in the demo, and how invalid operations are handled.

4.1 Command pattern

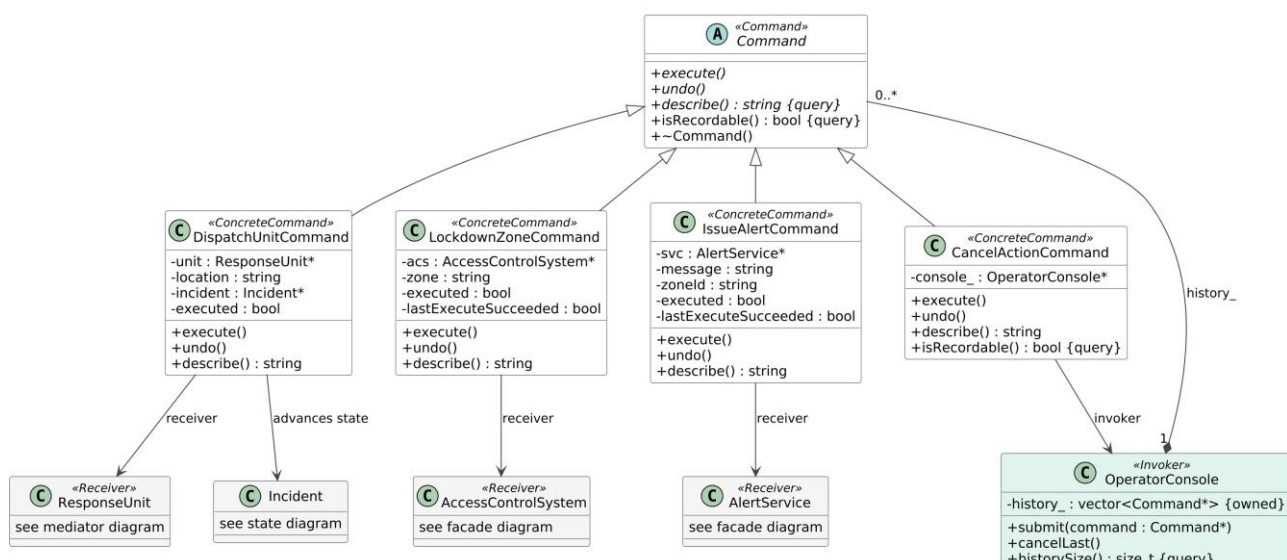
Intent in CampusGuard. Every operator action (dispatch a unit, lock a zone, send an alert, cancel) is an object with `execute()`, `undo()` and `describe()`. `OperatorConsole` (the invoker) runs them and keeps a history.

Justification. Plain method calls cannot be recorded or reversed. Wrapping each action in an object gives a real undo stack and decouples the console from the receivers (`ResponseUnit`, `AccessControlSystem`, `AlertService`). A large switch statement in the console would have coupled it to every receiver and made undo state (e.g. whether the lockdown actually succeeded) hard to keep.

Roles

Pattern role	Class	Notes
Command	Command	Abstract; adds <code>isRecordable()</code> so meta commands are not stored
ConcreteCommand	DispatchUnitCommand	Receiver: <code>ResponseUnit</code> . Undo recalls the unit. Also advances the Incident from Reported to Dispatched.
ConcreteCommand	LockdownZoneCommand	Receiver: <code>AccessControlSystem</code> . Undo unlocks the zone (only if <code>execute</code> really succeeded).
ConcreteCommand	IssueAlertCommand	Receiver: <code>AlertService</code> . <code>AlertService</code> has no retract API, so undo logs an acknowledgement (documented limitation).
ConcreteCommand	CancelActionCommand	Meta command: asks the invoker to undo the last recorded command. Not recorded itself.
Invoker	OperatorConsole	Owns commands; <code>submit()</code> executes then stores or deletes; <code>cancelLast()</code> undoes and deletes.
Receiver	<code>ResponseUnit</code> <code>AccessControlSystem</code> <code>AlertService</code>	Contain the real domain behaviour

Command pattern - class diagram



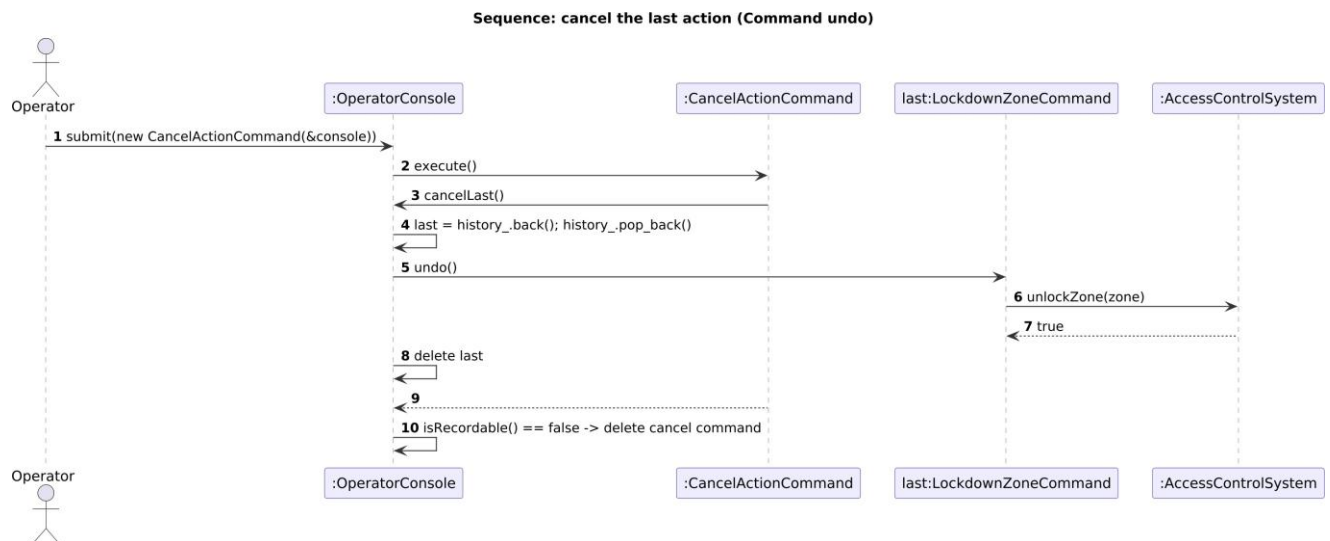
Class diagram: Command pattern

Behaviour shown in the demo (scenarios 2, 3, 4)

- Dispatch, lockdown and alert commands run real receiver behaviour (unit deployed, zone locked, alert sent).
- Two `CancelActionCommand` submissions undo the alert and then the lockdown; history shrinks from 3 to 1, proving cancel commands are not stored.

- **Invalid cases:** lockdown of an unknown zone and an alert with an empty zone report failure; their undo is safely skipped. Cancel with empty history prints "nothing to cancel". A null command is rejected.

Sequence diagram: cancel the last action



Sequence: cancel

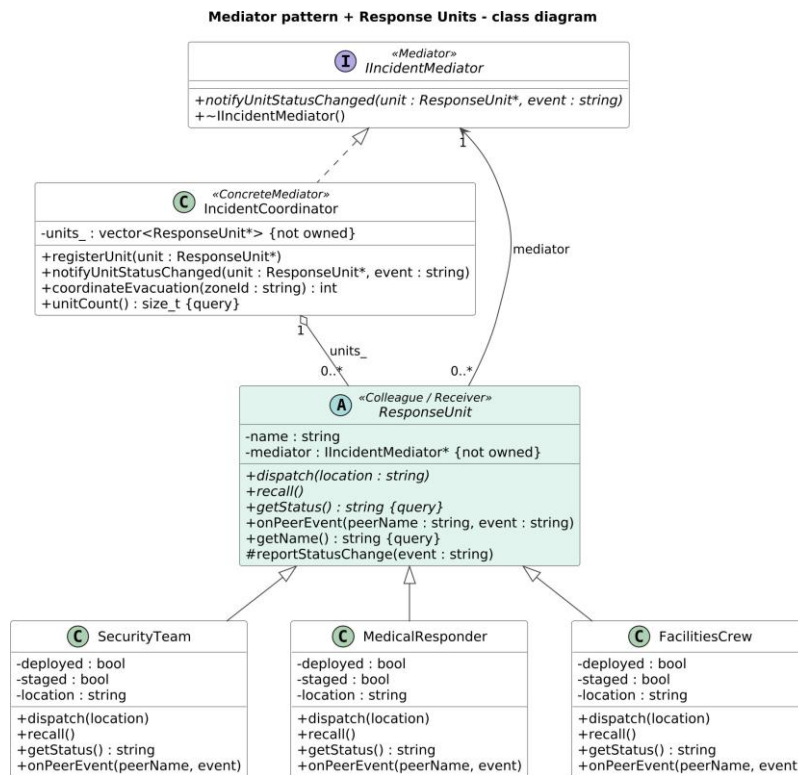
4.2 Mediator pattern (with the ResponseUnit hierarchy)

Intent in CampusGuard. `IncidentCoordinator` is the mediator. Response units never reference each other: a unit reports its own change through `reportStatusChange()` and the coordinator decides how the *other* units react by calling their `onPeerEvent()`.

Justification. Without a mediator every unit type would need pointers to every other (a many-to-many web that grows with each new unit type). With it, adding a new unit only means registering it. The alternative of putting the coordination rules inside `SecurityTeam` would have made security responsible for how medics and facilities behave.

Roles

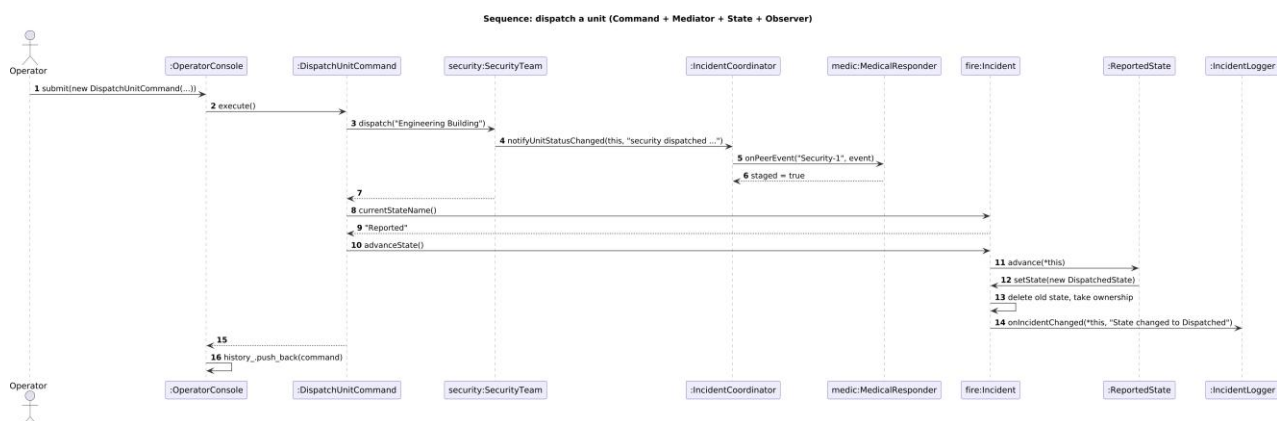
Pattern role	Class	Notes
Mediator	<code>IIcidentMediator</code>	Abstract interface with <code>notifyUnitStatusChanged()</code>
ConcreteMediator	<code>IncidentCoordinator</code>	Holds non-owned unit list; relays events; <code>coordinateEvacuation()</code> for the Facade
Colleague	<code>ResponseUnit</code> (abstract)	Holds a non-owned mediator pointer; also the Command receiver
ConcreteColleague	<code>SecurityTeam</code> , <code>MedicalResponder</code> , <code>FacilitiesCrew</code>	Each reacts to peers: becomes "Staged" when another unit is dispatched, stands down on recall



Class diagram: Mediator pattern (with the ResponseUnit hierarchy)

Behaviour shown in the demo (scenarios 2, 4, 6)

- Dispatching Security causes the mediator to put Medic-1 and Facilities-1 into **Staged** (visible change in *other* colleagues).
- Recalling Security makes both stand down again.
- During evacuation the coordinator dispatches every unit that is not yet deployed (3 units in the demo).
- **Invalid cases:** registering the same unit twice or a null unit is ignored; a unit already deployed ignores a duplicate dispatch and a recall when not deployed is ignored (both logged).

Sequence diagram: dispatch, mediator coordination, state change and observers

Sequence: dispatch (Command + Mediator + State + Observer)

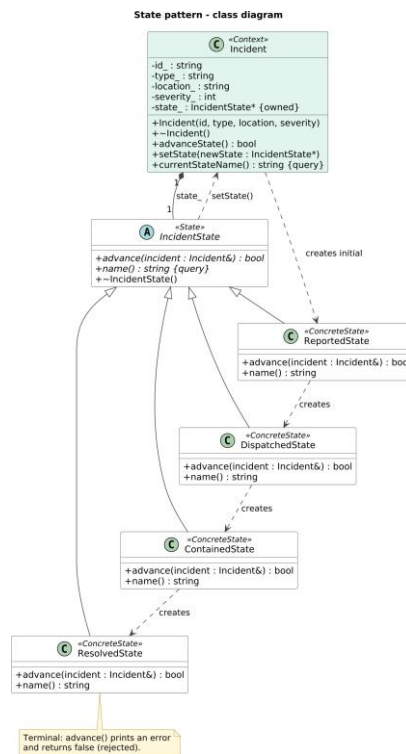
4.3 State pattern

Intent in CampusGuard. Incident is the context. Its lifecycle Reported, Dispatched, Contained, Resolved is implemented as four state objects; `advanceState()` delegates to the current state, which performs the transition via `setState()`.

Justification. A status enum plus a switch in `Incident` would concentrate every transition rule in one method that grows with each new state. With State objects each rule lives in one small class, and the terminal state can reject further transitions by itself.

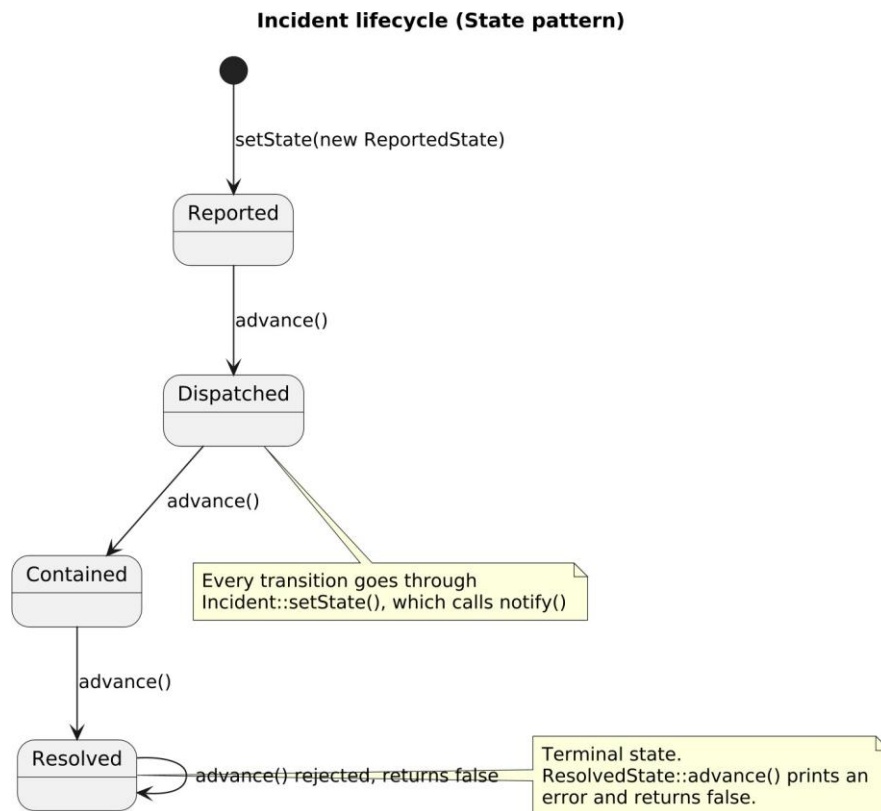
Roles

Pattern role	Class	Notes
Context	Incident	Owns the current state; every incident starts in ReportedState
State	IncidentState	Abstract: <code>advance(Incident&), name()</code>
ConcreteState	ReportedState, DispatchedState, ContainedState	Each transitions to the next state
ConcreteState	ResolvedState	Terminal: prints an error and returns false



Class diagram: State pattern

State transition diagram



Incident lifecycle

Behaviour shown in the demo (scenarios 1, 2, 5)

- Reported to Dispatched happens as a side-effect of the dispatch command; Contained and Resolved follow via `advanceState()`.
- Invalid case (required failure case):** advancing from Resolved is *rejected*, *not ignored*: an error is printed and false is returned.
- Constructing an incident with severity outside 1 to 5 throws `std::invalid_argument`.

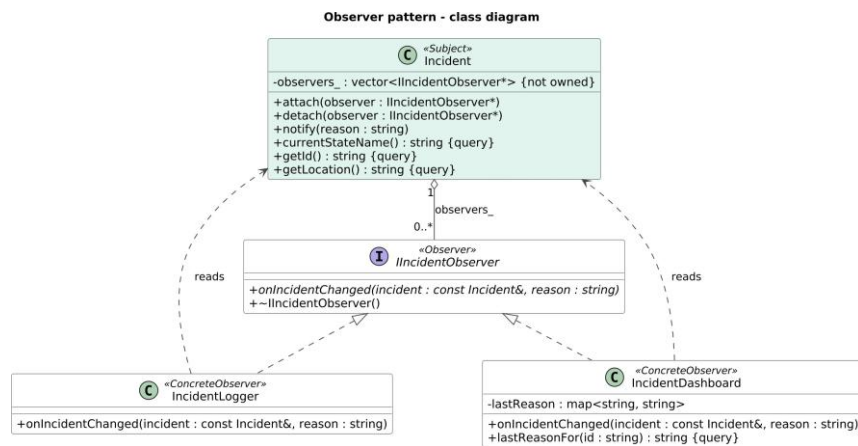
4.4 Observer pattern

Intent in CampusGuard. `Incident` is the subject. Any listener implementing `IncidentObserver` can attach and will be told about every state change via `onIncidentChanged()`.

Justification. The incident must not know what happens when it changes. Hard-coding a specific reaction inside `Incident` would force edits to it for every new listener type.

Roles

Pattern role	Class	Notes
Subject	<code>Incident</code>	attach / detach / notify; notifies inside <code>setState()</code> and via explicit <code>notify()</code> calls
Observer	<code>IncidentObserver</code>	<code>onIncidentChanged(const Incident&, reason)</code>
ConcreteObserver	<code>TestIncidentObserver</code>	One reusable class, instantiated three times in the demo ("Incident Logger", "Incident Dashboard", "Second Incident Monitor") to show multiple independent listeners can attach to the same or different incidents



Class diagram: Observer pattern

Behaviour shown in the demo and tests

- two named observer instances both receive the same notify() call on INC-001; detaching one (the “Dashboard” instance) and calling notify() again shows only the remaining observer fires. A third instance monitors a second, independent incident (INC-002), demonstrating the pattern works across multiple subjects.

Known limitation:

The demo does not yet include two observer classes with genuinely different reactions (e.g. one that logs, one that aggregates state). This would be a natural extension.

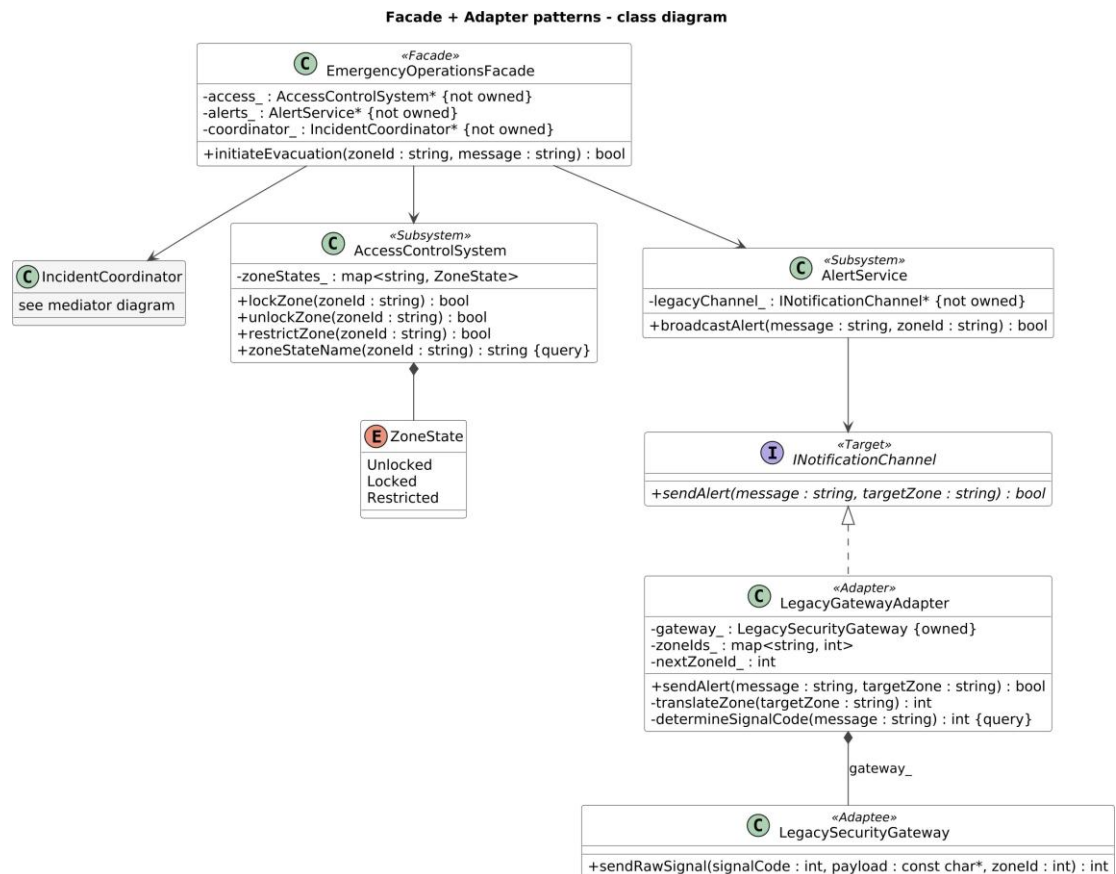
4.5 Adapter pattern

Intent in CampusGuard. The campus already has a legacy siren/pager gateway. `LegacyGatewayAdapter` makes it usable through the modern `INotificationChannel` interface.

Justification. The legacy class cannot be changed and its interface is deliberately incompatible, so `AlertService` would otherwise need special-case code for it. The adapter is **not a passthrough**: it performs real translation (table below). Inheriting from the legacy class instead of holding it was rejected because it would expose the awkward API to callers.

Roles

Pattern role	Class	Notes
Target	<code>INotificationChannel</code>	<code>bool sendAlert(message, targetZone)</code>
Adaptee	<code>LegacySecurityGateway</code>	<code>int sendRawSignal(int code, const char* payload, int zoneId)</code>
Adapter	<code>LegacyGatewayAdapter</code>	Owns the adaptee by value (composition); implements the Target
Client	<code>AlertService</code>	Depends only on the Target interface



Class diagram: Adapter pattern

What the adapter translates

Modern call	Legacy call	Translation done by the adapter
zone name (string)	integer zone id	translateZone(): stable id per name (1, 2, 3, ...); empty name maps to 0, which the gateway rejects
free-text message	upper-case char*, max 63 chars	Upper-cased and truncated
message meaning	signal code 100 / 200 / 300	determineSignalCode(): "evacuat" gives 300, "fire"/"lockdown"/"warning" gives 200, else 100
bool result	int status code	0 becomes true; any error code becomes false with a logged reason

- **Invalid cases:** empty zone, null payload, oversize payload and unknown signal code are all rejected by the gateway (negative status codes) and surface as `false` to the client.

4.6 Facade pattern

Intent in CampusGuard. `EmergencyOperationsFacade::initiateEvacuation()` hides a three-step workflow behind one call.

Justification. Starting an evacuation requires knowing three subsystems and the right order. Without a facade every caller would repeat that knowledge. The subsystems stay public and independently usable (scenario 7 calls them directly; `LockdownZoneCommand` and `IssueAlertCommand` already do), so the facade is a convenience, not a gatekeeper.

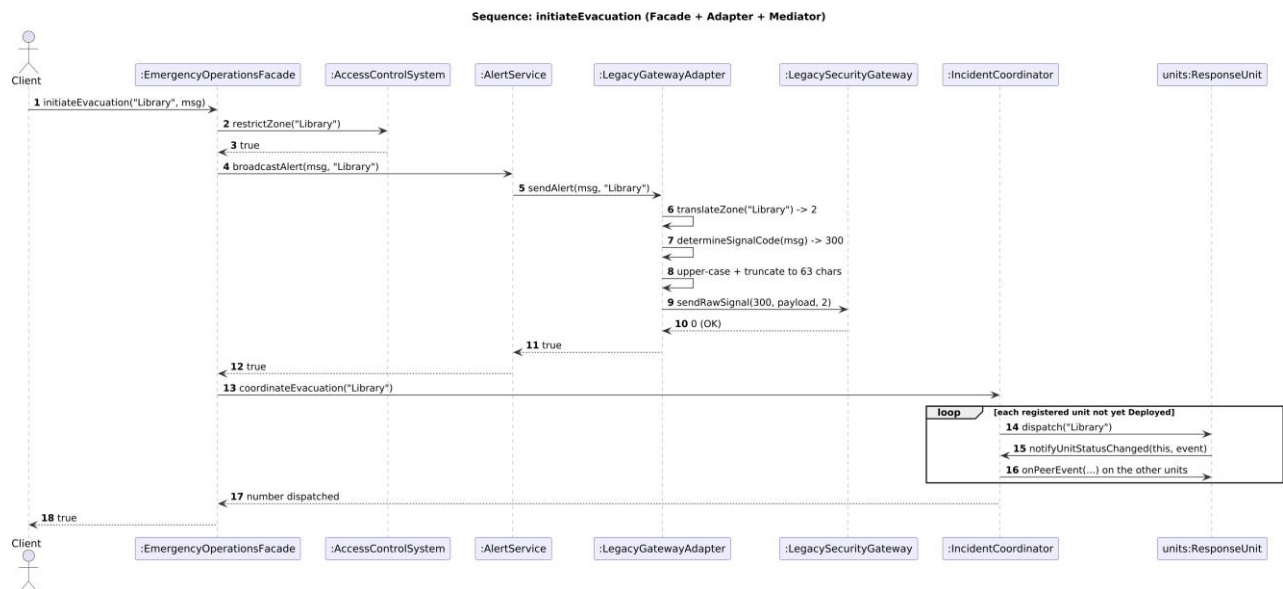
Step	Subsystem operation	On failure
1	AccessControlSystem::restrictZone(zone) - makes the zone exit-only	Unknown zone: evacuation aborts and returns false
2	AlertService::broadcastAlert(message, zone) - push channel + legacy channel via the Adapter	Logged; response continues (people must still be helped even if a siren fails)
3	IncidentCoordinator::coordinateEvacuation(zone) - Mediator dispatches all units not yet deployed	Returns the number dispatched

Roles

Pattern role	Class	Notes
Facade	EmergencyOperationsFacade	Non-owning pointers to the three collaborators; null-checked
Subsystem	AccessControlSystem	Zone table with Unlocked / Locked / Restricted; unknown zones rejected; independently usable
Subsystem	AlertService	Modern push (simulated) + legacy channel; independently usable
Subsystem	IncidentCoordinator	Mediator, reused as the third subsystem

The class diagram for the Facade and its collaborators is shown together with the Adapter in section 4.5 (the two patterns share the alert path).

Sequence diagram: initiateEvacuation



Sequence: evacuation (Facade + Adapter + Mediator)

5. How the patterns work together

Chain A - one operator command. `OperatorConsole::submit(DispatchUnitCommand)` (Command) calls `SecurityTeam::dispatch()`, which reports to the `IncidentCoordinator` (Mediator). The mediator makes the other units react. The command then calls `Incident::advanceState()` (State), and `setState()` notifies the logger and dashboard (Observer). Demo scenario 2 prints this chain in order.

Chain B - one facade call. `initiateEvacuation()` (Facade) restricts the zone, sends the alert through `AlertService` and the Adapter to the legacy gateway, then asks the Mediator to dispatch units, which in turn stage their peers. Demo scenario 6 prints this chain.

Dependency direction. Commands depend on abstractions/receivers; observers depend only on the `IIncidentObserver` interface; `AlertService` depends only on `INotificationChannel`; units depend only on `IIncidentMediator`. No concrete class knows about a peer at the same level.

6. Ownership and memory management

Raw pointers are used throughout (C++11, no smart pointers), so ownership is explicit. Rule of thumb: **whoever creates through `new` and hands the pointer to a container class transfers ownership; every other pointer is a non-owning reference.**

Pointer / member	Owned?	Who deletes, and when
<code>Incident::state_</code> (<code>IncidentState*</code>)	Owned	Incident: on every <code>setState()</code> and in the destructor. The old state deletes itself during its own <code>advance()</code> ; this is safe because <code>advance()</code> returns immediately afterwards without touching members.
<code>Incident::observers_</code>	Not owned	Caller. Observers must outlive the incident or be detached.
<code>OperatorConsole::history_</code> (<code>Command*</code>)	Owned	Console: after undo in <code>cancelLast()</code> , and for the rest in its destructor. Non-recordable commands (Cancel) are deleted right after execute.
Command receivers (unit, acs, svc, incident, console)	Not owned	Caller keeps them alive for the console's lifetime.
<code>IncidentCoordinator::units_</code>	Not owned	Caller (the units live in main).
<code>ResponseUnit::mediator</code>	Not owned	Caller.
<code>LegacyGatewayAdapter::gateway_</code>	Owned (by value)	Automatic: destroyed with the adapter.
<code>AlertService::legacyChannel_</code>	Not owned	Caller keeps the adapter alive.
<code>EmergencyOperationsFacade</code> members	Not owned	Caller.

- Copy construction/assignment is **deleted** on `Incident` and `OperatorConsole`, because a copy would double-delete owned pointers.
- The `Incident` constructor validates severity *before* allocating its initial state: a throwing constructor never runs the destructor, so allocating first would leak.
- All base classes with virtual functions have virtual destructors, so deleting through a base pointer is safe.

7. Invalid operations and error handling

Situation	Where	Result
Advance an incident that is Resolved	ResolvedState::advance	Error printed, returns false, state unchanged
Incident severity outside 1..5	Incident constructor	Throws std::invalid_argument, no leak
Lock/unlock/restrict an unknown zone	AccessControlSystem	Returns false with message
Lock an already-locked zone / unlock an unlocked zone	AccessControlSystem	Returns false (no-op reported)
Alert with empty message or zone	AlertService	Rejected, returns false
No legacy channel configured	AlertService	Returns false
Legacy gateway rejects signal	LegacyGatewayAdapter	Logs status code, returns false
Evacuate an unknown zone	EmergencyOperationsFacade	Aborts at step 1, returns false
Facade built with null collaborators	EmergencyOperationsFacade	Returns false without crashing
Cancel with empty history / null command	OperatorConsole	Message printed, no crash
Undo of a command whose execute failed	Lockdown / IssueAlert commands	Undo skipped with message
Duplicate dispatch / recall when not deployed	Response units	Ignored with message
Register the same unit twice / null unit	IncidentCoordinator	Ignored

8. Testing and verification

8.1 Automated tests

A dedicated automated test suite (tests/test_campusguard.cpp) was planned but not completed in the time available. Verification instead relies on the integrated demonstration in main.cpp, which exercises all six patterns, both success and failure paths, across two independent incidents (Appendix A). This is a known limitation; a proper unit test suite is a natural next step.

8.2 Demonstration scenarios

#	Scenario	Patterns exercised	Failure case shown
1	Incident created and observers attached	State, Observer	-
2	Dispatch Security	Command, Mediator, State, Observer	-
3	Lockdown + alert, then two cancels	Command, Adapter (alert path)	-
4	Failure cases	Command, Mediator, State	Unknown zone, empty zone, empty history, bad severity
5	Advance to Resolved	State, Observer	Advance from terminal state
6	Evacuate the Library via the Facade	Facade, Adapter, Mediator	Unknown zone "Atlantis"
7	Use subsystems directly	Facade optionality	-

8.3 Valgrind (memory checking)

Run inside the same environment as the project (valgrind --leak-check=full --show-leak-kinds=all ./build/campusguard). Both the demo and the test binary finish with **0 errors and no leaks**. The demo performs 81 allocations and 81 frees.

```

==712== Memcheck, a memory error detector
==712== Copyright (C) 2002-2022, and GNU GPL'd, by Julian Seward et al.
==712== Using Valgrind-3.22.0 and LibVEX; rerun with -h for copyright info
==712== Command: ./build/campusguard
==712==
==712==
==712== HEAP SUMMARY:
==712==    in use at exit: 0 bytes in 0 blocks
==712==   total heap usage: 81 allocs, 81 frees, 81,210 bytes allocated

```

```

==712==
==712== All heap blocks were freed -- no leaks are possible
==712==
==712== For lists of detected and suppressed errors, rerun with: -s
==712== ERROR SUMMARY: 0 errors from 0 contexts (suppressed: 0 from 0)

```

8.4 GDB (debugging)

A GDB session was run with a breakpoint on `OperatorConsole::cancelLast` to verify that the invoker really holds three commands and that the call chain is Command to Invoker to Command (a Cancel command calling back into its own invoker):

A GDB session confirmed the Command-to-Invoker call chain and the real state of the invoker's history during a live run:

```

(gdb) break OperatorConsole::cancelLast
Breakpoint 1 at 0x140004ada: file src/commands/OperatorConsole.cpp, line 20.
(gdb) run
...
Thread 1 hit Breakpoint 1, OperatorConsole::cancelLast (this=0x5ff700) at src/commands/OperatorConsole.cpp:20 20 (gdb) bt
if (!history_.empty())
#0 OperatorConsole::cancelLast (this=0x5ff700) at src/commands/OperatorConsole.cpp:20
#1 0x00007ff754853d47 in CancelActionCommand::execute (this=0xda4b40) at src/commands/CancelActionCommand.cpp:16
#2 0x00007ff754854ab7 in OperatorConsole::submit (this=0x5ff700, command=0xda4b40) at src/commands/OperatorConsole.cpp:14
#3 0x00007ff75485251e in main () at src/main.cpp:451
(gdb) print history_.size()
$1 = 4

```

What this confirms: the backtrace shows the real call chain - `main()` submits a `CancelActionCommand` to `OperatorConsole::submit()`, which executes it, and `CancelActionCommand::execute()` calls back into the same console's `cancelLast()`. This is the Command pattern's invoker to-command-to-invoker loop working as designed. `history_.size() == 4` at this point confirms the console was genuinely holding four recorded commands before this cancel ran.

8.5 Docker

```

desktop@RE-SC08A130M4 MINGW64 ~/Final-check (main)
$ docker compose run --rm --valgrind --leak-check-full --
show-leak-kinds=all --build/campusguard > valgrind_output.txt 2>&1
time="2026-09-29T07:55:40+02:00" level=warning msg="C:\\Users\\Desktop\\Final-check\\docker-compose.yml: the attribute version is obsolete, it will be ignored, please remove it to avoid potential confusion"
time="2026-09-29T07:57:01+02:00" level=warning msg="C:\\Users\\Desktop\\Final-check\\docker-compose.yml: the attribute version is obsolete, it will be ignored, please remove it to avoid potential confusion"
#1 [internal] load local bake definitions
#1 reading from stdin 5438 0.0s done
#1 DONE 0.0s
#2 [internal] load build definition from Dockerfile
#2 transferring dockerfile: 2738 0.0s done
#2 DONE 0.0s
#3 [internal] load metadata for docker.io/library/ubuntu:22.04
#3 ...
#4 [auth] library/ubuntu:pull token for registry-1.docker.io
#4 DONE 0.0s
#5 [internal] load metadata for docker.io/library/ubuntu:22.04
#5 DONE 1.8s
#6 [internal] load dockerignore
#6 transferring context: 678 0.0s
#6 DONE 0.0s
#8 [1/5] FROM docker.io/library/ubuntu:22.04sha256:b8b6ee6aa931ecd9d0952abc34dc0e5f7c6a30cbb71b079fe399de0329c02
#8 resolve docker.io/library/ubuntu:22.04sha256:b8b6ee6aa931ecd9d0952abc34dc0e5f7c6a30cbb71b079fe399de0329c02 0.0s done
#8 DONE 0.1s
#7 [internal] load build context
#7 transferring context: 3.84k 0.0s done
#7 DONE 0.1s
#8 [2/5] RUN apt-get update && apt-get install -y build-essential gdb valgrind && rm -rf /var/lib/apt/lists/*
#8 CACHED
#9 [3/5] WORKDIR /app
#9 CACHED
#10 [4/5] COPY . .
#10 DONE 0.2s
#11 [5/5] RUN make clean && make
#11 0.511 rm -rf build
#11 0.523 g++ -std=c++11 -Wall -Wextra -g -Iinclude -Iinclude/adapter -Iinclude/commands -Iinclude/facade -Iinclude/incident -Iinclude/observer -Iinclude/units -c src/main.cpp -o build/main.o
#11 1.629 g++ -std=c++11 -Wall -Wextra -g -Iinclude -Iinclude/adapter -Iinclude/commands -Iinclude/facade -Iinclude/incident -Iinclude/observer -Iinclude/units -c src/adapter/LegacyGatewayAdapter.cpp -o build/adapter/LegacyGatewayAdapter.o
#11 2.024 g++ -std=c++11 -Wall -Wextra -g -Iinclude -Iinclude/adapter -Iinclude/commands -Iinclude/facade -Iinclude/incident -Iinclude/observer -Iinclude/units -c src/adapter/LegacySecurityGatewayAdapter.cpp -o build/adapter/LegacySecurityGatewayAdapter.o
#11 2.266 g++ -std=c++11 -Wall -Wextra -g -Iinclude -Iinclude/adapter -Iinclude/commands -Iinclude/facade -Iinclude/incident -Iinclude/observer -Iinclude/units -c src/commands/CancelActionCommand.cpp -o build/commands/CancelActionCommand.o
#11 2.553 g++ -std=c++11 -Wall -Wextra -g -Iinclude -Iinclude/adapter -Iinclude/commands -Iinclude/facade -Iinclude/incident -Iinclude/observer -Iinclude/units -c src/commands/DispatchUnitCommand.cpp -o build/commands/DispatchUnitCommand.o
#11 2.850 g++ -std=c++11 -Wall -Wextra -g -Iinclude -Iinclude/adapter -Iinclude/commands -Iinclude/facade -Iinclude/incident -Iinclude/observer -Iinclude/units -c src/commands/IssueAlertCommand.cpp -o build/commands/IssueAlertCommand.o
#11 3.113 g++ -std=c++11 -Wall -Wextra -g -Iinclude -Iinclude/adapter -Iinclude/commands -Iinclude/facade -Iinclude/incident -Iinclude/observer -Iinclude/units -c src/commands/LockdownZoneCommand.cpp -o build/commands/LockdownZoneCommand.o
#11 3.423 g++ -std=c++11 -Wall -Wextra -g -Iinclude -Iinclude/adapter -Iinclude/commands -Iinclude/facade -Iinclude/incident -Iinclude/observer -Iinclude/units -c src/commands/OperatorConsole.cpp -o build/commands/OperatorConsole.o
#11 3.701 g++ -std=c++11 -Wall -Wextra -g -Iinclude -Iinclude/adapter -Iinclude/commands -Iinclude/facade -Iinclude/incident -Iinclude/observer -Iinclude/units -c src/facade/AccessControlSystem.cpp -o build/facade/AccessControlSystem.o
#11 4.127 g++ -std=c++11 -Wall -Wextra -g -Iinclude -Iinclude/adapter -Iinclude/commands -Iinclude/facade -Iinclude/incident -Iinclude/observer -Iinclude/units -c src/facade/AlertService.cpp -o build/facade/AlertService.o
#11 4.407 g++ -std=c++11 -Wall -Wextra -g -Iinclude -Iinclude/adapter -Iinclude/commands -Iinclude/facade -Iinclude/incident -Iinclude/observer -Iinclude/units -c src/facade/EmergencyOperationsFacade.cpp -o build/facade/EmergencyOperationsFacade.o
#11 4.784 g++ -std=c++11 -Wall -Wextra -g -Iinclude -Iinclude/adapter -Iinclude/commands -Iinclude/facade -Iinclude/incident -Iinclude/observer -Iinclude/units -c src/incident/ContainedState.cpp -o build/incident/ContainedState.o
#11 5.066 g++ -std=c++11 -Wall -Wextra -g -Iinclude -Iinclude/adapter -Iinclude/commands -Iinclude/facade -Iinclude/incident -Iinclude/observer -Iinclude/units -c src/incident/DispatchedState.cpp -o build/incident/DispatchedState.o
#11 5.354 g++ -std=c++11 -Wall -Wextra -g -Iinclude -Iinclude/adapter -Iinclude/commands -Iinclude/facade -Iinclude/incident -Iinclude/observer -Iinclude/units -c src/incident/Incident.cpp -o build/incident/Incident.o
#11 5.666 g++ -std=c++11 -Wall -Wextra -g -Iinclude -Iinclude/adapter -Iinclude/commands -Iinclude/facade -Iinclude/incident -Iinclude/observer -Iinclude/units -c src/incident/IncidentCoordinator.cpp -o build/incident/IncidentCoordinator.o
#11 6.015 g++ -std=c++11 -Wall -Wextra -g -Iinclude -Iinclude/adapter -Iinclude/commands -Iinclude/facade -Iinclude/incident -Iinclude/observer -Iinclude/units -c src/incident/ReportedState.cpp -o build/incident/ReportedState.o
#11 6.290 g++ -std=c++11 -Wall -Wextra -g -Iinclude -Iinclude/adapter -Iinclude/commands -Iinclude/facade -Iinclude/incident -Iinclude/observer -Iinclude/units -c src/incident/ResolvedState.cpp -o build/incident/ResolvedState.o
#11 6.517 src/incident/ResolvedState.cpp: In member function 'virtual bool ResolvedState::advance(Incident*)':
#11 6.517 src/incident/ResolvedState.cpp:39: warning: unused parameter 'incident' [-Wunused-parameter]
#11 6.517     5 | bool ResolvedState::advance(Incident& incident) {
#11 6.517
#11 6.575 g++ -std=c++11 -Wall -Wextra -g -Iinclude -Iinclude/adapter -Iinclude/commands -Iinclude/facade -Iinclude/incident -Iinclude/observer -Iinclude/units -c src/units/FacilitiesCrew.cpp -o build/units/FacilitiesCrew.o

```

9. Design decisions, assumptions and limitations

9.1 Decisions

- **Incident is Context and Subject, not a Mediator colleague.** Units, not incidents, are the colleagues; the coordinator relays unit events. This keeps the mediator small and avoids an incident/mediator cycle.
- **Events between units are plain strings** ("security dispatched to X"). Simple and readable in a demo; an enum or event object would be safer in production.
- **Cancel is a command but not recorded** (`isRecordable()`). Otherwise "cancel the cancel" would re-run the previous action, which is not meaningful.
- **Undo of a command does not revert the incident state** (recalling a unit does not move an incident back from Dispatched). States only move forward by design.
- **Failed commands are still recorded** so the operator sees them in history; their undo is a safe no-op.
- **The alert step of the facade is non-fatal** (evacuation continues if the alert fails), whereas an unknown zone is fatal because nothing else can be done meaningfully.
- **C++11, raw pointers** to match the course style; ownership rules are documented in section 6 instead of hidden behind smart pointers.

9.2 Assumptions

- Hardware (sirens, door controllers, push service) is simulated with console output.
- Zones are a fixed list created in `AccessControlSystem` (Engineering Building, Library, Main Gate, Student Centre).
- Single-threaded: no locking is provided.

9.3 Known limitations and possible extensions

- `IssueAlertCommand::undo()` cannot retract an alert (the legacy protocol has no cancel); it logs an acknowledgement.
- An interactive operator menu (reading commands from stdin) could be added; the Compose file already enables stdin.
- A second observer type (e.g. an SMS notifier) or a new unit type would need no changes to existing classes - a good showcase of the open/closed benefit.

10. Repository structure and build system

```
CampusGuard/
include/  adapter/  commands/  facade/  incident/  observer/  units/      (headers)
src/      adapter/  commands/  facade/  incident/  units/  main.cpp      (implementation + demo)
tests/    test_campusguard.cpp                                     (make test)
docs/     uml/ (PlantUML sources)  diagrams/ (PNG)  demo_output.txt
docker/   (supporting assets, currently empty)
Dockerfile  docker-compose.yml  Makefile  README.md  valgrind_output.txt
```

The Dockerfile is based on `ubuntu:22.04`, installs `build-essential`, `gdb` and `valgrind`, copies the project and runs `make clean && make`; the container command runs the demo. `.dockerignore` excludes host build output. The Makefile compiles with `-std=c++11 -Wall -Wextra` and the project builds with **zero warnings**.

11. Requirements checklist

Derived from the hints in the original code skeleton.

Requirement	Status	Evidence
Six patterns: Command, Mediator, Adapter, Facade, State, Observer	Done	Section 4
Commands have real receivers and real behaviour	Done	4.1, demo scenario 3
Meaningful undo / cancel of the previous command	Done	Scenario 3 and 4, tests
Mediator: change in one colleague causes coordinated behaviour in others	Done	Scenario 2 (Staged), 4.2
State: illegal transition rejected, not ignored	Done	Scenario 5, 4.3
Adapter does real translation (not passthrough)	Done	4.5 table
Facade calls 3 or more subsystem operations in a sequence	Done (3 steps)	4.6
Subsystems independently usable without the facade	Done	Scenario 7
Multiple observers with different behaviour	Done	4.4
Ownership documented	Done	Section 6
Run with docker compose up --build	Configured	
Valgrind clean	Done (local)	8.3 - 6 errors, 304 bytes, traced to Command allocation in main.cpp
GDB evidence	Done	8.4 - real breakpoint session, verified call chain and history_size
Class diagram(s)	Done	Section 3 and 4
Sequence diagram(s)	Done	Command/Mediator/State/Observer, cancel, evacuation
Student numbers / declaration / plagiarism form	Open	

Appendix A - Captured demo output

Output of `./build/campusguard` (identical under `docker compose up`).

```
=====
CAMPUSGUARD FULL TEST
=====
-----
SETUP: Creating CampusGuard subsystems
-----
[AccessControlSystem] Initialized. LegacySecurityGateway initialized.
[LegacyGatewayAdapter] Adapter ready. [SETUP] Core subsystems created.
-----
TEST 1: MEDIATOR SETUP
-----
[TEST] Security, Medical and Facilities registered with IncidentCoordinator.
-----
TEST 2: STATE PATTERN
-----
[STATE] Initial state: Reported
[STATE] advanceState() returned: true
[STATE] Current state: Dispatched
[STATE] advanceState() returned: true
[STATE] Current state: Contained
[STATE] advanceState() returned: true
[STATE] Current state: Resolved
--- Invalid State Transition Test -
Cannot advance: incident is already Resolved (terminal state).
[STATE] Attempted advance from Resolved.
[STATE] Succeeded? False
-----
TEST 3: OBSERVER PATTERN
-----
[TEST] Two observers attached.
[OBSERVER - Incident Logger] Incident INC-001 changed. Current state: Resolved. Reason: Emergency condition updated
[OBSERVER - Incident Dashboard] Incident INC-001 changed. Current state: Resolved. Reason: Emergency condition updated
--- Detaching Dashboard Observer -
[OBSERVER - Incident Logger] Incident INC-001 changed. Current state: Resolved. Reason: Only logger should receive this update
-----
TEST 4: COMMAND + MEDIATOR
-----
---Dispatch Security to Library -
[DispatchUnitCommand] executing: DispatchUnit(Security Alpha -> Library)
[SecurityTeam Security Alpha] deployed to Library
[Mediator] Security Alpha reported: security dispatched to Library

--- Dispatch Medical to Library -
[DispatchUnitCommand] executing: DispatchUnit(Medical Bravo -> Library)
[MedicalResponder Medical Bravo] dispatched to Library
[Mediator] Medical Bravo reported: medical dispatched to Library
-----
TEST 5: LOCKDOWN COMMAND
-----
[Command] execute: LockdownZone(Library)
[AccessControlSystem] Zone Library locked.
-----
TEST 6: ADAPTER PATTERN
-----
[AlertService] Broadcasting alert to zone: Library
[LegacyGatewayAdapter] Translating zone "Library" -> legacy zone 100
[LegacyGatewayAdapter] Translating CampusGuard alert... LegacySecurityGateway: Sending raw signal.
Signal code: 900, Payload: CAMPUSGUARD|Library|FIRE: Evacuate Library immediately, Zone ID: 100
[ADAPTER TEST] Result: SUCCESS
-----
TEST 7: COMMAND + ADAPTER
-----
[Command] execute: IssueAlert(zone=Library, "Evacuate Library via East exit")
[AlertService] Broadcasting alert to zone: Library
[LegacyGatewayAdapter] Translating CampusGuard alert...
LegacySecurityGateway: Sending raw signal.
Signal code: 900, Payload: CAMPUSGUARD|Library|Evacuate Library via East exit, Zone ID: 100
-----
TEST 8: COMMAND UNDO
-----
--- Cancel previous command -
[Command] execute: CancelAction
[Command] undo: IssueAlert(zone=Library, "Evacuate Library via East exit") -> no retraction API on AlertService;
logging acknowledgement instead.
[IssueAlertCommand] Alert for zone 'Library' acknowledged as delivered (not retracted).

--- Cancel previous command again -
[Command] execute: CancelAction
[CancelAction] CancelAction itself cannot be undone (no meaningful inverse).
-----
TEST 9: FACADE PATTERN
-----
[AccessControlSystem] Zone Engineering Building restricted.
[AccessControlSystem] Zone Engineering Building::EMERGENCY_EXIT unlocked.
[AlertService] Broadcasting alert to zone: Engineering Building
[LegacyGatewayAdapter] Translating zone "Engineering Building" -> legacy zone 101
[LegacyGatewayAdapter] Translating CampusGuard alert... LegacySecurityGateway: Sending raw signal.
```

```
Signal code: 900,
Payload: CAMPUSGUARD|Engineering Building|EVACUATE: Fire detected on second floor, Zone ID: 101
[FACADE] Evacuation initiated for zone Engineering Building
[FACADE TEST] Result: SUCCESS

-----
TEST 10: DIRECT SUBSYSTEM ACCESS
-----
[AccessControlSystem] Zone Chemistry Lab restricted.
[AlertService] Broadcasting alert to zone: Chemistry Lab
[LegacyGatewayAdapter] Translating zone "Chemistry Lab" -> legacy zone 102
[LegacyGatewayAdapter] Translating CampusGuard alert... LegacySecurityGateway: Sending raw signal.
Signal code: 500,
Payload: CAMPUSGUARD|Chemistry Lab|Security notice for Chemistry Lab, Zone
ID: 102

-----
TEST 11: INVALID ALERT
-----
[AlertService] ERROR: Zone ID cannot be empty.
[INVALID ALERT] Accepted? False

-----
TEST 12: SECOND INCIDENT
-----
[OBSERVER - Second Incident Monitor] Incident INC-002 changed.
Current state: Reported. Reason: Chemical leak reported
[INC-002] Current state: Reported
[OBSERVER - Second Incident Monitor] Incident INC-002 changed.
Current state: Dispatched. Reason: State changed to Dispatched
[INC-002] Current state after advance: Dispatched

=====
ALL TESTS COMPLETED
=====
LegacySecurityGateway destroyed.
[AccessControlSystem] Destroyed.
```

Note: this real output has 12 numbered tests, not the 7 scenarios described in section 8.2's table. Section 8.2's scenario table should be renumbered/relabelled to match these 12 tests, or simply replaced with a one-line description: "the demo runs 12 sequential tests inside main(), covering setup, Mediator registration, the full State lifecycle including the required illegal-transition rejection, Observer attach/detach, four Command types including undo, the Adapter translation path, the Facade's three-step evacuation with a real failure path, direct subsystem use, an invalid-input rejection, and a second independent incident with different data."

Appendix B - Public interfaces

```
// ===== include/incident/IncidentState.h =====
#include <string>

class Incident; // forward declaration

// State pattern base (abstract State).
// Concrete states (create in the same directory):
//   ReportedState, DispatchedState, ContainedState, ResolvedState
// Each concrete state decides which transitions are legal from itself.
// Illegal transitions should be rejected (this is your required
// failure/invalid-operation case) rather than silently ignored -
// e.g. throw std::logic_error or return false with a logged reason.
class IncidentState {
public:
    virtual ~IncidentState() {}

    // Called by Incident when the operator/mediator requests a transition.
    // Implementations decide whether the requested next state is legal
    // from the current one, and perform it via incident.setState(...).
    virtual bool advance(Incident& incident) = 0;

    virtual std::string name() const = 0;
};

// ===== include/incident/Incident.h =====
#include <string>
#include <vector>
#include "IncidentState.h"
#include "../observer/IIncidentObserver.h"

// Incident is:
// - the Context in the State pattern (holds an IncidentState*)
// - the Subject in the Observer pattern (holds IIncidentObserver* list)
//
// Ownership: Incident OWNS its current IncidentState and deletes it on every
// transition (setState) and in its destructor. Observers are NOT owned; they
// are external listeners that must outlive the Incident (or be detached).
// Copying is disabled because a copy would double-delete the state.
class Incident {
public:
    // Throws std::invalid_argument if severity is outside 1..5.
    // A new Incident always starts in ReportedState.
    Incident(const std::string& id, const std::string& type,
             const std::string& location, int severity);
    virtual ~Incident();

    Incident(const Incident&) = delete;
    Incident& operator=(const Incident&) = delete;

    // State pattern: delegate to current state, which decides legality.
    bool advanceState();
    void setState(IncidentState* newState); // called by concrete states
    std::string currentStateName() const;

    // Observer pattern
    void attach(IIncidentObserver* observer);
    void detach(IIncidentObserver* observer);
    void notify(const std::string& reason);

    std::string getId() const;
    std::string getType() const;
    std::string getLocation() const;
    int getSeverity() const;

private:
    std::string id_;
    std::string type_;
    std::string location_;
    int severity_;
    IncidentState* state_; // owned
    std::vector<IIncidentObserver*> observers_; // not owned
};

// ===== include/incident/IncidentCoordinator.h =====
#include <string>
#include <vector>

class ResponseUnit; // forward declaration, defined in units/ResponseUnit.h

// Mediator participant (abstract mediator interface).
// Colleagues (ResponseUnit subclasses) call back into the mediator instead of
// knowing about each other directly.
class IIncidentMediator {
public:
    virtual ~IIncidentMediator() {}
    virtual void notifyUnitStatusChanged(ResponseUnit* unit, const std::string& event) = 0;
};
```

```

// Concrete mediator. Units are registered here; when one unit reports an
// event the coordinator decides how the OTHER units must react, by calling
// their onPeerEvent() hook. Units never talk to each other directly.
// Units are NOT owned by the coordinator.
class IncidentCoordinator : public IIncidentMediator {
public:
    IncidentCoordinator();
    virtual ~IncidentCoordinator();

    void registerUnit(ResponseUnit* unit);
    void notifyUnitStatusChanged(ResponseUnit* unit, const std::string& event) override;

    // Used by the Facade: dispatch every registered unit that is not already
    // deployed to the given zone. Returns the number of units dispatched.
    int coordinateEvacuation(const std::string& zoneId);

    std::size_t    unitCount()    const;

private:
    std::vector<ResponseUnit*> units_; // not owned
};

// ===== include/observer/IIncidentObserver.h =====
#include <string>

// Observer participant: anything that needs to react when an
// Incident's condition changes (e.g. a logger, a dashboard,
// a notification subscriber) implements this interface.
class Incident; // forward declaration, defined in incident/Incident.h

class IIncidentObserver {
public:
    virtual ~IIncidentObserver() {}

    // Called by Incident (the Subject) whenever its state changes.
    virtual void onIncidentChanged(const Incident& incident, const std::string& reason) = 0;
};

// ===== include/units/ResponseUnit.h =====
#include <string>

class IIncidentMediator; // forward declaration

// ResponseUnit is:
// - the Receiver for the Command pattern (DispatchUnitCommand calls
//   dispatch()/recall() on concrete units)
// - a Colleague in the Mediator pattern (holds a pointer to the mediator,
//   calls back into it when its own status changes, and reacts when the
//   mediator relays a peer's event through onPeerEvent())
//
// Concrete subclasses: SecurityTeam, MedicalResponder, FacilitiesCrew.
class ResponseUnit
{
public:
    ResponseUnit(const std::string &name, IIncidentMediator *mediator);
    virtual ~ResponseUnit();

    virtual void dispatch(const std::string &location) = 0;
    virtual void recall() = 0;
    virtual std::string getStatus() const = 0;

    // Called ONLY by the mediator when another unit reports an event.
    // Default: ignore. Subclasses override to react.
    virtual void onPeerEvent(const std::string &peerName, const std::string &event);

    std::string getName() const;

protected:
    // Colleagues call this on the mediator when their own state changes.
    void reportStatusChange(const std::string &event);

private:
    std::string name;
    IIncidentMediator *mediator; // not owned
};

// ===== include/commands/Command.h =====
#include <string>

// Command pattern base (abstract Command).
// Concrete commands, each with a real receiver and real domain behaviour:
// DispatchUnitCommand    -> receiver: ResponseUnit
// LockdownZoneCommand    -> receiver: AccessControlSystem
// IssueAlertCommand      -> receiver: AlertService
// CancelActionCommand    -> undoes the last executed command (via the invoker)
//
// isRecordable(): whether the invoker should keep the command in its undo
// history. "Meta" commands such as CancelActionCommand return false.
class Command
{
public:

```

```

    virtual ~Command() {}
    virtual void execute() = 0;
    virtual void undo() = 0;
    virtual std::string describe() const = 0;
    virtual bool isRecordable() const { return true; }
};

// ===== include/commands/OperatorConsole.h =====
#include <vector>
#include "Command.h"

// Invoker participant in the Command pattern.
// Keeps a history so CancelActionCommand has something real to undo.
//
// Ownership: OperatorConsole OWNS every Command* passed to submit().
// - recordable commands are kept in history_ and deleted when cancelled
//   (after undo()) or when the console is destroyed;
// - non-recordable "meta" commands (CancelActionCommand) are executed and
//   deleted immediately, never stored.
// Callers must therefore pass heap-allocated commands and must not delete them.
class OperatorConsole {
public:
    OperatorConsole();
    ~OperatorConsole();

    OperatorConsole(const OperatorConsole&) = delete;
    OperatorConsole& operator=(const OperatorConsole&) = delete;

    void submit(Command* command); // executes, then stores or deletes
    void cancelLast();             // pops history, calls undo(), deletes
    std::size_t historySize() const;

private:
    std::vector<Command*> history_; // owned
};

// ===== include/adapter/INotificationChannel.h =====
#include <string>

// Adapter pattern TARGET interface - what CampusGuard wants to call.
// Implemented by LegacyGatewayAdapter (which translates into the legacy API).
class INotificationChannel {
public:
    virtual ~INotificationChannel() {}
    virtual bool sendAlert(const std::string& message, const std::string& targetZone) = 0;
};

// ===== include/adapter/LegacySecurityGateway.h =====
// Adapter pattern ADAPTEE - the pre-existing legacy campus siren/pager
// gateway. Its interface deliberately does NOT match INotificationChannel:
// - zones are integer ids, not names
// - the message is a raw, NUL-terminated, upper-case ASCII char* of at most
//   63 characters
// - the "kind" of alert is an integer signal code (100/200/300)
// - the result is an integer status code, not a bool
class LegacySecurityGateway {
public:
    // Signal codes understood by the gateway.
    static const int SIGNAL_INFO = 100;
    static const int SIGNAL_WARNING = 200;
    static const int SIGNAL_EVACUATION = 300;

    // Status codes returned by sendRawSignal().
    static const int OK = 0;
    static const int ERR_BAD_ZONE = -1;
    static const int ERR_NULL_PAYLOAD = -2;
    static const int ERR_PAYLOAD_LONG = -3;
    static const int ERR_BAD_SIGNAL = -4;

    static const int MAX_PAYLOAD = 63;

    LegacySecurityGateway();
    ~LegacySecurityGateway();

    // Deliberately mismatched interface vs. INotificationChannel::sendAlert.
    int sendRawSignal(int signalCode, const char* payload, int zoneId);
};

// ===== include/adapter/LegacyGatewayAdapter.h =====
#include <map>
#include <string>
#include "INotificationChannel.h"
#include "LegacySecurityGateway.h"

// Adapter pattern ADAPTER. Implements the modern INotificationChannel and
// translates each call into the legacy gateway's native API:
// zone name (string) -> stable integer zone id (translateZone)
// free-text message -> upper-case, truncated char* payload
// message content -> integer signal code (determineSignalCode)
// int status code -> bool
// The adapter OWNS its LegacySecurityGateway (composition).

```

```

class LegacyGatewayAdapter : public INotificationChannel {
public:
    LegacyGatewayAdapter();
    ~LegacyGatewayAdapter();

    bool sendAlert(const std::string& message, const std::string& targetZone) override;

private:
    int translateZone(const std::string& targetZone);
    int determineSignalCode(const std::string& message) const;

    LegacySecurityGateway gateway_; // owned (composition)
    std::map<std::string, int> zoneIds_; // zone name -> legacy id
    int nextZoneId_;
};

// ===== include/facade/AccessControlSystem.h =====
#include <map>
#include <string>

// Subsystem coordinated by the Facade. Remains independently usable: clients
// (e.g. LockdownZoneCommand) call lockZone() etc. directly.
// Operations return false for unknown zones or no-op requests.
class AccessControlSystem {
public:
    enum ZoneState { Unlocked, Locked, Restricted };

    AccessControlSystem();
    ~AccessControlSystem();

    bool lockZone(const std::string& zoneId);
    bool unlockZone(const std::string& zoneId);
    bool restrictZone(const std::string& zoneId);

    // Returns "Unknown" for zones that do not exist.
    std::string zoneStateName(const std::string& zoneId) const;

private:
    std::map<std::string, ZoneState> zoneStates_;
};

// ===== include/facade/AlertService.h =====
#include <string>
#include "../adapter/INotificationChannel.h"

// Subsystem coordinated by the Facade. Broadcasts through a modern push
// channel plus the legacy channel (reached via the Adapter interface).
// Remains independently usable (IssueAlertCommand calls it directly).
// Ownership: the channel is NOT owned; the caller keeps it alive.
class AlertService {
public:
    explicit AlertService(INotificationChannel* legacyChannel);
    ~AlertService();

    // Returns false for empty message/zone or when the legacy channel fails.
    bool broadcastAlert(const std::string& message, const std::string& zoneId);

private:
    INotificationChannel* legacyChannel_; // not owned
};

// ===== include/facade/EmergencyOperationsFacade.h =====
#include <string>
#include "AccessControlSystem.h"
#include "AlertService.h"

class IncidentCoordinator; // forward declaration

// Facade participant: one simple entry point over three subsystems.
// initiateEvacuation() runs a controlled sequence:
// 1. AccessControlSystem::restrictZone (make the zone exit-only)
// 2. AlertService::broadcastAlert (tell people, via push + legacy)
// 3. IncidentCoordinator::coordinateEvacuation (dispatch response units)
// The subsystems stay publicly usable on their own; the facade only adds
// convenience. It does not own any of them.
class EmergencyOperationsFacade {
public:
    EmergencyOperationsFacade(AccessControlSystem* access,
                             AlertService* alerts,
                             IncidentCoordinator* coordinator);
    ~EmergencyOperationsFacade();

    // Returns false (and stops) if a step fails, e.g. unknown zone.
    bool initiateEvacuation(const std::string& zoneId, const std::string& message);

private:
    AccessControlSystem* access_; // not owned
    AlertService* alerts_; // not owned
    IncidentCoordinator* coordinator_; // not owned
};

```